National University of Computer and Emerging Sciences, Lahore Campus

	Course Name:	Design and Analysis of Algorithms	Course Code:	CS 302
	Degree Program:	BS-CS	Semester:	Fall 2019
	Exam Duration:	180 Minutes	Total Marks:	60
	Paper Date:	Tuesday, 17 December 2019	Weight	
	Section:	ALL	Page(s):	12
	Exam Type:	Final Exam		

Student : Name:_____ **Roll No.**_____ **Section:**_____

Instruction/Notes: Rough Sheets can be used but please write your answers in the space provided.

ATTEMPT ALL QUESTIONS

[SHORT QUESTIONS]

SH 1. [2 Points]

The order-of-growth of the running time of one algorithm is n^2; the order-of-growth of the running time of a second algorithm is n log n. List two compelling reasons why a programmer might still prefer to use the n^2 algorithm instead of the n log n one?

SH 2. [1 + 1 + 2 Points]

Let G = (V; E) be a simple graph with n vertices. Suppose the weight of every edge of G is one.

(a) What is the weight of a minimum spanning tree of G?

(b) Suppose we change the weight of one of the edges of G to 1/2. What is the weight of the minimum spanning tree of G?

(c) Suppose we change the weight of k < V edges of G to 1/2. What is the minimum and maximum possible weights for the minimum spanning tree of G?

SH 3. [6 Points]
For each of the following algorithms, give tight upper bounds on the worst case and best case running times for an input of size N.

ALGORITHM	Best case Running Time	Worst case Running Time
RADIX Sort		
QUICK Sort		
INSERTION Sort		
Algorithm FUN_ONE (Input of size N) FOR I = 1 to N Do Constant_Amount_of_Processing; return ANSWER;		
Algorithm FUN_TWO (Input of size N) FOR C = 1 to N FOR D = 1 to C Do Constant_Amount_of_Processing; return ANSWER;		
Algorithm FUN_THREE (Input of size N) IF N == 0 Return 0 ELSE CALL FUN_THREE (Input of size N/2) CALL FUN_THREE (Input of size N/2) CALL FUN_ONE (Input of size N) Return ANS END IF		

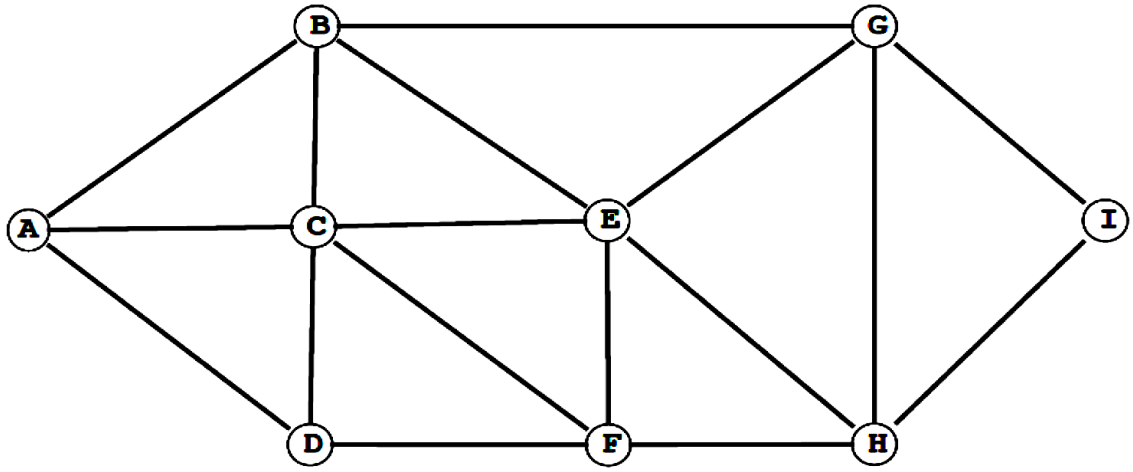
SH 4. [3 Points]

Following three code segments have been taken from the implementations of three sorting algorithms in a language like C++. Your job in this problem is to identify the correct sorting algorithm by viewing only part of the code (i.e. for each algorithm most of the code is not visible). If there are multiple possible answers then write all possible algorithms and if the code is not that of a sorting algorithm then write NONE as your answer

CODE	ALGORITHM NAME
void FUN_SORT (int A[], int N){ **// SOME CODE NOT VISIBLE** if(A[j] > A[j + 1]) **{** int T = A[j]; A[j] = A[j+1]; A[j+1] = T; **}** **// SOME CODE NOT VISIBLE** **}**	
void FUN_SORT (int A[], int S, int E){ if(S == E) return; **// SOME CODE NOT VISIBLE** M = (S + E)/2; FUN_SORT(A, S, M); FUN_SORT(A, M + 1, E); **// SOME CODE NOT VISIBLE** **}**	
void FUN_SORT (int A[], int N){ **// SOME CODE NOT VISIBLE** S = A[I]; J = I - 1; while(J >= 0 && A[j] > S){ A[J+1] = A[J]; J = J -1 } A[J+ 1] = S; **// SOME CODE NOT VISIBLE** **}**	

SH 5. [3 Points]
Run breadth-first search on the graph below, starting at vertex A.

Assume the adjacency lists are in sorted order, e.g., when exploring vertex F, the algorithm considers the edge F - C before F - D, F - E, or F - H.

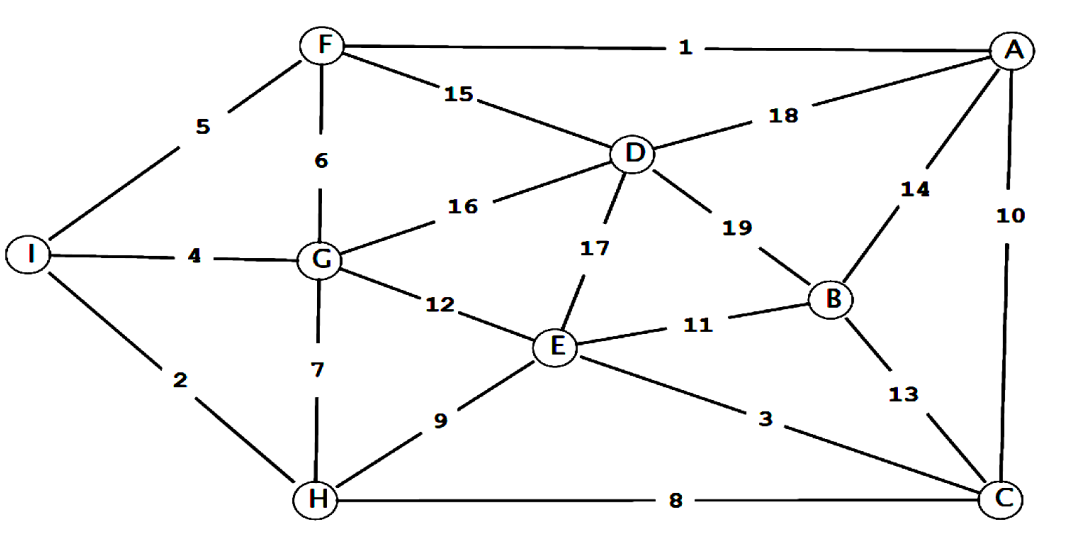


List the vertices in the order in which the vertices are enqueued.

A B

--- --- --- --- --- --- --- --- ---

SH 6. [5 Points]
Consider the following graph with 9 vertices and 19 edges with weights from 1 to 19



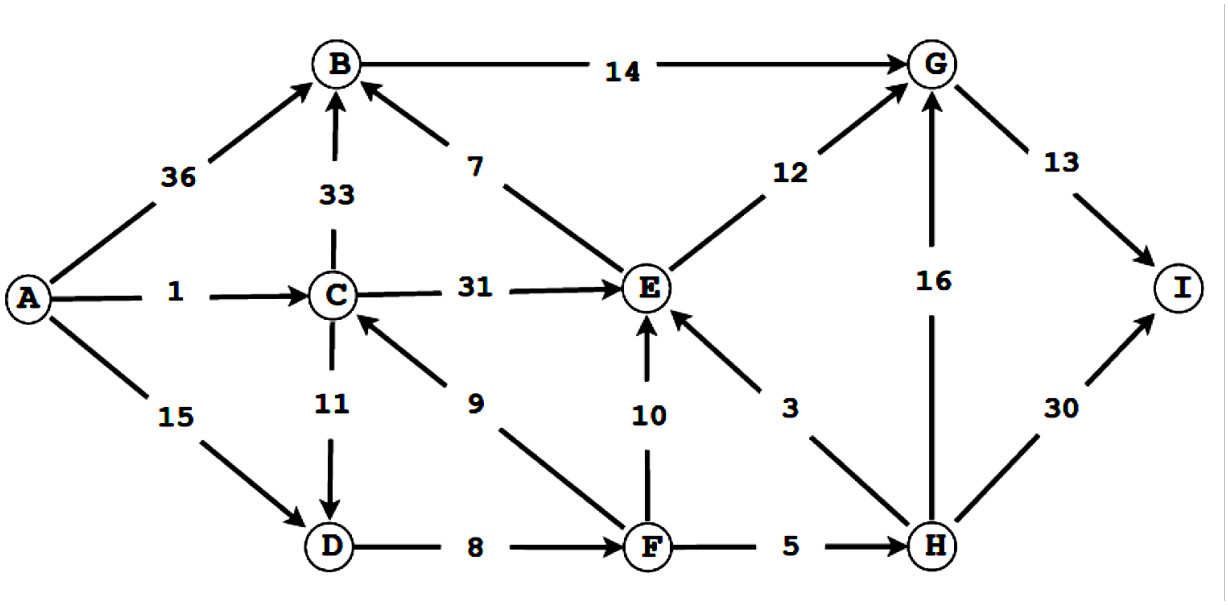
Complete the sequence of edges in the MST in the order that Kruskal's algorithm includes them (by specifying their edge weights).

1

--- --- --- --- --- --- --- ---

SH 7. [3 + 2 Points]

Run Dijkstra's algorithm on the weighted directed graph below, starting at vertex A.



(a) List the vertices in the order in which the vertices are dequeued (for the first time) from the priority queue and give the length of the shortest path from A.

Vertex	A	C							
Distance	0	1	___	___	___	___	___	___	___

(b) Also mark the edges in the shortest path tree on the figure above.

SH 8 [3 + 2 Points]

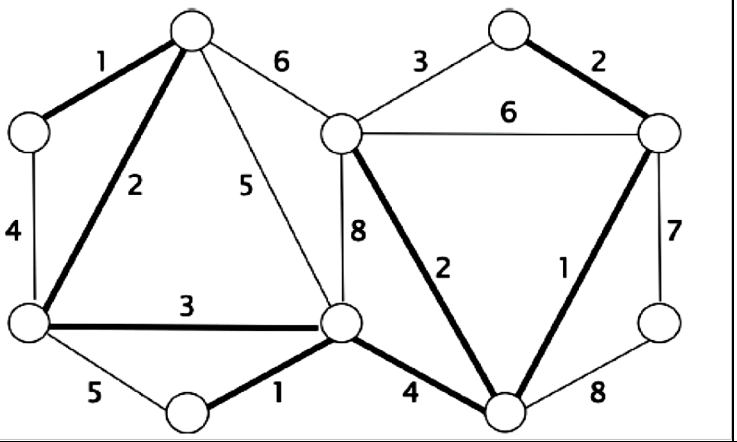
Describe, how can we use a finalized implementation of single-source shortest path algorithm (**SSSP**) to

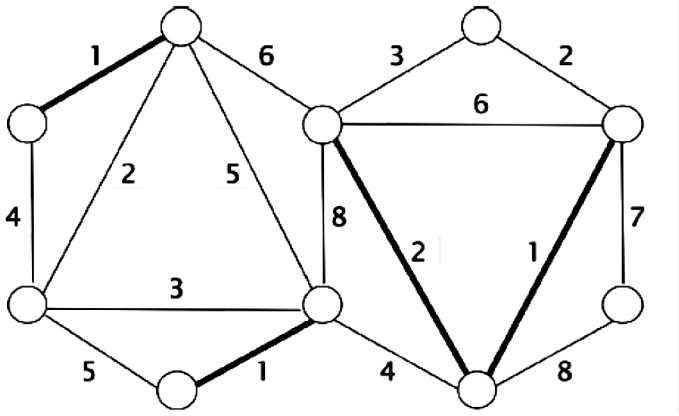
Find single-destination shortest path without changing **SSSP?**

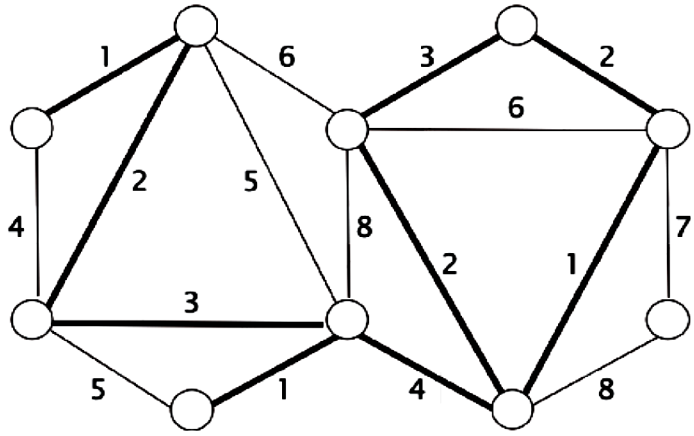
Find all pairs shortest path without changing **SSSP?**

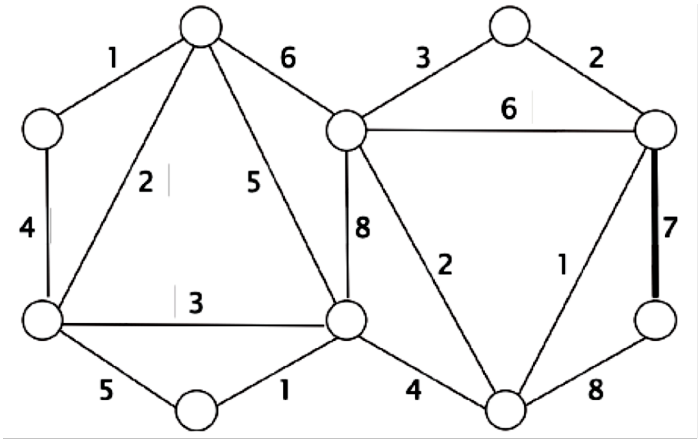
SH 9. [5 Points]

A set of edges is highlighted in different copies of the same graph in each of the figures below. Determine whether it could possibly represent a partial spanning tree obtained from (a prematurely stopped) Prim's algorithm, (a prematurely stopped) Kruskal's algorithm, both, or neither. Write "Prim's", "Kruskal's", "Both", or "Neither" the blank space below each figure.









[Algorithm Design]

Problem 1 [A Vacation to the Strange Island] [6 Points]

Strange Island is a beautiful place, but it is strange. There are many destinations on the island, including parks, gardens, cinemas, shopping plazas etc. The roads between these destinations are one-way. The strange thing is that there might exist places A and B on this island such that it is possible to go from A to B, but no way to get back from B to A.

You have been given a map of the strange island, which is a directed graph with the destinations as the vertices, and the one-way roads as the edges. You're staying at a hotel on vertex **S**. You're planning to go out and see some destinations, but you also want to come back to your hotel at night. You want to make these decisions quickly, so you decide to write a program/algorithm, to do it for you. Your algorithm, **safeDestinations** must mark all those destinations on the island that are safe to visit i.e. there is a way from S to the destination and also a way from the destination to S.

You are free to use any algorithm covered in class as a subroutine in **safeDestinations.**

The asymptotically fastest solution will get full credit. Other solutions will get partial credit depending upon their correctness.

[BLANK PAGE]

Problem 2 [Bottles and Caps] [6 Points]

You have information about a set of n bottles stored in an array **B** and a set of n caps stored in an array **C**. You may assume that one cap fits exactly one bottle only. However, these two arrays are not in any particular order, i.e. cap C[k] may not be the cap that fits the bottle B[k]. Moreover, there is no way to directly compare the sizes of two bottle with each other, or two caps with each other, so we cannot sort bottles and caps separately.

The only operation we can use is a function called **FITS**. It receives a bottle and a cap, and returns 0, if the cap fits the bottle, -1 if the cap is too small for the bottle, and 1 if the cap is too large for the bottle.

Using this function, design an algorithm that arranges the arrays B and C in such a way that cap C[k] fits the bottle B[k] for all indices k = 1, 2,…n. Your algorithm should be asymptotically fastest possible.

Hint: A partitioning algorithm like that of Quick Sort might be a possible solution

Problem 3 [Dynamic Programming] [5 + 5 Points]

While reading chapter 15 of my favorite book on algorithms I came across the following recursive formulation of a secret problem. The formulation was used to compute the value of **m[1, n]** using values stored in an array **p[0,… n]** and an important design principle called Dynamic Programming.

$$m[i,j] = \begin{cases} 0 & \text{if } i = j\ , \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j\} & \text{if } i < j\ . \end{cases} \qquad (15.7)$$

Part a) Write a recursive algorithm **with memoization** that computes the required value using the values stored in the array **p** and a 2D array **m**. It can be assumed that the arrays **p** and **m** are globally available. Hence the prototype function **computeSecretValue (i , j)** can be used.

Part b) It might be a surprise for some of us that not all languages support recursive functions. Therefore, we need a non-recursive bottom-up implementation of the function in part b.

Write that function assuming that arrays **p** and **m** are available globally.